

Project Report: Real-Time Edge Pothole Detection

Submitted for the ARM Bharat AI SoC Challenge

Submitted by: - Vemana Venkata Pujithram

- Venkata Subhash

- Souhrud Reddy

1. Abstract

This report details the development and deployment of a highly optimized, real-time object detection system designed to identify road anomalies (potholes) using edge computing hardware. The primary objective was to deploy a robust AI model on a Raspberry Pi 4 Model B while maintaining a strict balance between detection accuracy and hardware-accelerated inference speed. By leveraging INT8 quantization, multi-threading, and optimal camera buffer management, the system successfully surpassed the minimum requirement of 5 FPS, achieving near-real-time inference entirely on an Arm-based CPU.

2. Methodology

The project was executed in two distinct phases: model training in a high-compute environment, and deployment/inference on an edge device.

2.1. Model Selection and Training

The core architecture chosen for this task was YOLOv5, selected for its strong baseline performance in real-time object detection. The model was trained on a custom dataset consisting of various road pothole images. Initial training and validation were conducted using the PyTorch framework to establish a high baseline of accuracy.

2.2. Edge Environment Configuration

To ensure a clean, reproducible, and conflict-free environment on the Raspberry Pi (running Debian Bookworm/Trixie), the pixi package manager was utilized. This allowed for granular control over dependencies, specifically resolving architectural conflicts by locking the environment to Python 3.10 and numpy < 2.0, ensuring flawless compatibility with the TensorFlow Lite runtime.

3. Hardware Utilization & Optimization Techniques

Running raw PyTorch models on a Raspberry Pi is computationally prohibitive. To achieve the ≥ 5 FPS target, multiple layers of optimization were applied directly to the hardware and software stack:

3.1. INT8 Quantization

The most significant optimization was the conversion of the base PyTorch model into a TensorFlow Lite (.tflite) format using full Integer-8 (INT8) quantization. By downcasting the 32-bit floating-point weights to 8-bit integers, the model size was reduced by over 70%. More importantly, this allowed the Arm Cortex-A72 CPU to process the neural network mathematically using highly efficient integer arithmetic, drastically reducing memory bandwidth bottlenecks.

3.2. Resolution Scaling

High-resolution inputs overwhelm edge CPUs. The input tensor of the quantized model was scaled to a 320x320 resolution. The OpenCV (cv2) pipeline was configured to aggressively downscale incoming frames from the V4L2 USB camera before passing them to the AI, ensuring compute cycles were not wasted on unnecessary pixel density.

3.3. CPU Multi-Threading

By default, the TFLite interpreter utilizes a single CPU thread. To fully saturate the Raspberry Pi 4's hardware, the inference engine was explicitly initialized with `num_threads=4`. This distributed the tensor operations across all four physical cores of the processor, maximizing throughput.

3.4. Input Lag Mitigation

To prevent the system from processing outdated frames—a common issue in live OpenCV streams—the `cv2.CAP_PROP_BUFFERSIZE` was explicitly set to 1. This forced the hardware to drop stale frames and only pass the absolute latest image to the inference engine, eliminating ghosting and input latency.

3.5. Non-Maximum Suppression (NMS)

To maintain high precision in the visual output, a programmatic Non-Maximum Suppression filter was implemented with an Intersection Over Union (IOU) threshold of 0.45. This ensured that overlapping bounding boxes generated by the quantized model were merged, presenting a single, confident detection per pothole.

4. Results & Performance

The system was tested under live conditions via a LAN-connected remote desktop environment. The deployed solution demonstrated exceptional stability and performance metrics:

- **Accuracy (mAP):** The model achieved a Mean Average Precision of 0.878, demonstrating high reliability in differentiating potholes from standard road textures.
- **Inference Speed:** The optimized pipeline achieved a sustained processing rate of ~5.6 FPS on the Raspberry Pi 4 CPU. This successfully exceeds the challenge's strict requirement for near-real-time detection (≥ 5 FPS).

5. Conclusion

This project successfully demonstrates the viability of deploying advanced computer vision models on low-power, Arm-based edge devices. Through rigorous quantization, strategic hardware threading, and strict memory management, the resulting system operates independently, in real-time, and with high accuracy. The methodologies outlined in this report provide a scalable blueprint for deploying autonomous road anomaly detection systems in real-world, resource-constrained environments.